



TERRAFORM • INTERVIEW PREPARATION

INTERVIEW ARSENAL

110+ real questions · the decoder · red-flag answers · follow-up traps · 2 mock interviews with rubrics

Master V1 ✓
foundations

Master V2 ✓
production

Practice Lab ✓
40 drills

Interview Arsenal
← this book

The Arsenal, Mapped

01	How Interviews Actually Work	09	Z7 · Lifecycle & Guards (6)
02	The 30-Second Answers	10	Z8 · Multi-Env & Scale (6)
03	Z1 · Fundamentals (6)	11	Z9 · Cloud & Governance (6)
04	Z2 · State & Backends (8)	12	Z10 · War Stories Bank (8)
05	Z3 · Workflow & CLI (6)	13	Mock Interview 1 — Mid (8)
06	Z4 · Variables & Config (6)	14	Mock Interview 2 — Senior (8)
07	Z5 · Resources & Logic (6)	15	Rapid-Fire Round (24)
08	Z6 · Modules (6)	16	The Night Before

☐  Junior — define it

☐  Mid — mechanism & why

☐  Senior — architecture & disaster

 = asked constantly

How to March Through This Book

From [understanding the interrogation](#) → [calibrating your 30-second answers](#) → [arming every zone Z1–Z9](#) → [wargaming disasters](#) → [rehearsing full interviews aloud](#) → [sharpening with rapid fire](#) → [deploying the night before](#).

THE RULE

Safe gets you to the next question; senior ends the topic. Every question carries all three rungs — climb until the interviewer nods and moves on.

RED FLAGS

A red flag isn't wrong knowledge — it's wrong signal. Each question names the answers that damage you, and why they read badly from the hiring side.

How Terraform Interviews Work

Terraform interviews test three things in order: can you **define it**, can you **explain the mechanism**, and can you **survive the disaster**. Most candidates stop at rung one — this book rungs every question three high.

THE SHAPE OF THE INTERROGATION

Round 1 screens definitions (what is state, what does init do). Round 2 probes mechanisms (why a saved plan, what a lock protects). Round 3 replays disasters (dead CI holder, deleted state, +/- on a database). Safe answers pass rounds 1–2; only senior depth closes round 3.

1 The 3-rung decoder

Rung	What it is	What the interviewer hears
✓ Safe	Correct definition, 2 lines	"Knows the tool — ask me harder."
🚀 Senior depth	Mechanism + WHY	"Has run this in prod — topic closed."
🚩 Red flag	Confident wrong-signal answer	"Dangerous near prod — probe deeper."

HOW TO CLIMB

Open safe on every question. If the interviewer leans in, spend the senior rung. Never volunteer a red-flag-shaped sentence — each question below names yours so you can avoid it.

2 The 4 interviewer archetypes

- 📖 **The Definitionalist** — "Define state." Wants crisp sentences. Feed rung 1, stop. Rambling mechanisms here reads as bluffing.
- ⚙️ **The Mechanic** — "What happens when two applies collide?" Wants the mechanism (locks, bindings, plan files). This is rung 2 territory.
- 🔥 **The Arsonist** — "State deleted, prod passionate, go." Wants calm triage order: inspect, never assume, next plan heals. Never promise rollback.
- 🔍 **The Auditor** — "How does your team gate applies?" Wants PR plan-gate-apply, fmt -check, pinned providers, no committed secrets. Process answers only.

UNIVERSAL RED FLAGS

"We apply with -auto-approve by hand." "State is in git so everyone has it." "Force-unlock first, ask later." Any one of these can end an interview on its own.

The 30-Second Answers

Six answers you must deliver cold, in 30 seconds each. Say the **Say** line verbatim; the **Lands** line is why it works.

① What is Terraform?

Say: "Terraform is HashiCorp's tool for building, changing, and versioning infrastructure — I declare end-state in HCL, it figures out the API calls. Single Go binary, 2014, one workflow across clouds."

② Why not ClickOps?

Say: "Console infra is tribal knowledge — nobody can answer what prod is or who changed it Tuesday. In Terraform, git log answers both, and every change passes plan review first."

③ State in one breath?

Say: "State is Terraform's memory: a JSON map of config to real IDs, IPs, and dependencies. Config says what I want, state says what exists, refresh verifies — delete it and Terraform forgets everything."

④ Plan before apply?

Say: "Plan previews, apply executes — and apply on a saved plan file runs exactly what was reviewed, no drift window. Bare apply in prod is how outages happen."

⑤ Modules?

Say: "Modules package infra like functions — inputs vary per caller, outputs cross the boundary as plain values like module.vpc.vpc_id. Registry modules get version pins, or upstream edits you at 3 AM."

⑥ Floci?

Say: "Floci is a local AWS emulator — container on port 4566, dummy creds plus endpoint override. I ran init → plan → apply → destroy against it and proved idempotence with a clean second plan."

✓ REHEARSAL RULE

Record yourself. Any answer over 30 seconds is a rung-1 answer with rung-2 fat on it — trim, don't add.

What Terraform Is & Why It Exists

Q1 🔥 ? Terraform in one sentence — what is it?

✓ SAFE (JUNIOR → MID)

A tool for building, changing, and versioning infrastructure — declare what you want in config, Terraform makes reality match.

🚀 SENIOR DEPTH

HashiCorp, 2014, single Go binary with zero runtime. Manages existing providers (AWS, Cloudflare...) and custom in-house ones against any API — breadth is the whole point.

🚩 RED FLAG

"Config manager / AWS-only / container orchestrator" — that's Ansible, CloudFormation, K8s. Naming the wrong layer fails round 1 instantly.

Q2 🔥 ? Declarative vs imperative — which pair is correct?

✓ SAFE (JUNIOR → MID)

Declarative states WHAT (Terraform); imperative scripts HOW (playbooks).

🚀 SENIOR DEPTH

Terraform takes end-state and derives ordering plus API calls itself — you never write the steps. Playbooks list steps to execute, with no state to diff against.

🚩 RED FLAG

The inverted pair (Terraform=HOW). It's the exam's favorite trap shape — interviewers use it the same way.

Q3 🔥 ? Same config applied twice — second plan shows...? Why?

✓ SAFE (JUNIOR → MID)

0 to add, 0 to change, 0 to destroy — "No changes." Healthy steady state.

🚀 SENIOR DEPTH

Idempotence: same code → same infra, re-apply is a no-op by design. Refresh re-read reality, compare found zero drift: desired == known == real.

🚩 RED FLAG

"Terraform skips planning" — it planned fully, then did nothing. And "faster on repeat" confuses caching with idempotence.

Q4 ? AWS + Cloudflare + GitHub in one apply — why Terraform?

✓ SAFE (JUNIOR → MID)

Platform-agnostic: one apply across any providers the moment a second API enters.

🚀 SENIOR DEPTH

CloudFormation can't leave AWS; splitting tools loses the single plan. Speed, community, practices are all true of Terraform and all miss the question — scope is the answer.

🚩 RED FLAG

Answering "it's faster" or "great community" — true, irrelevant, and it tells a Mechanic you dodge scope questions.

Memory, Locks & Ledger Surgery

Q1 🔥 ? What does terraform.tfstate actually store?

✓ SAFE (JUNIOR → MID)

JSON memory mapping config to real objects: IDs, IPs, ARNs, CIDRs, dependency edges — what the APIs returned after apply.

🚀 SENIOR DEPTH

Four functions: track (what's mine), compare (desired vs actual → plan diff), metadata (values config never declared), perform (cache so plan skips re-querying).

🚩 RED FLAG

"Re-queried live every second, state optional" or hand-editing the JSON — every fix has a state command; hand edits corrupt.

Q2 🔥 ? Five engineers, one local state file — first fix?

✓ SAFE (JUNIOR → MID)

Remote backend. Local + two writers = last apply wins silently, then duplicates.

🚀 SENIOR DEPTH

S3 + DynamoDB, two jobs: versioned encrypted bucket stores, lock table guards. Backend checklist is all three — versioning + encryption + locking; one pillar is a half answer.

🚩 RED FLAG

Committing tfstate to git or emailing it around — state holds secrets in plaintext plus the keys to everything.

Q3 ? Sensitive values in state — how do you handle them?

✓ SAFE (JUNIOR → MID)

They sit in state — so state never enters git, backend is encrypted, access is restricted.

🚀 SENIOR DEPTH

Leak paths multiply: tfvars values, CI logs echoing outputs raw. Encrypt the backend, mark variables sensitive, keep secrets out of logs and repos.

🚩 RED FLAG

"Base64 is encryption" thinking, or terraform output -raw secrets into CI logs.

Q4 🔥 ? Second apply starts while the first holds the lock?

✓ SAFE (JUNIOR → MID)

It waits or fails with a lock error — coordinate, queue, serialize. A live lock means a live writer.

🚀 SENIOR DEPTH

Force under a LIVE holder corrupts state for both runners — the cardinal sin. Force is for the dead only; per-env lock tables isolate blast radius.

🚩 RED FLAG

"Force-unlock first, ask later." Uttering this near a live apply ends interviews.

The Loop: init → plan → apply

Q1 🔥 ? Fresh clone on a new laptop — first command, and what appears?

✓ SAFE (JUNIOR → MID)

terraform init — downloads provider binaries, installs modules, configures the backend. Nothing works until it runs.

🚀 SENIOR DEPTH

Binaries land in .terraform/ (ignored, re-fetchable); the lock file pins EXACT builds (committed). Bumping a constraint does nothing until init -upgrade — plan keeps the locked binary.

🚩 RED FLAG

Planning before init, or committing .terraform/ — megabytes of regret for zero reproducibility.

Q2 🔥 ? Core workflow in order?

✓ SAFE (JUNIOR → MID)

Write → init → plan → apply. First word is write — init needs something to install for.

🚀 SENIOR DEPTH

Plan is read-only and safe anytime — run it on every PR. Read the diff symbols: + add, ~ change, - destroy. A -/+ on a database means stop and snapshot.

🚩 RED FLAG

"Plan is optional, apply covers it" — apply-without-plan is the anti-pattern in prod.

Q3? Why apply a saved plan file instead of bare apply?

✓ SAFE (JUNIOR → MID)

apply tfplan executes EXACTLY the reviewed diff — what you saw is what runs, no drift window.

🚀 SENIOR DEPTH

PR → plan posted → merge → gated apply: infra gets code-review discipline. -out serializes any plan, including destroy previews (plan -destroy -out).

🚩 RED FLAG

Auto-apply on merge skipping human review — the pipeline judges what nobody read.

Q4? Apply interrupted halfway (Ctrl+C) — what is true?

✓ SAFE (JUNIOR → MID)

Partial stands — finished creates stay, state is written incrementally. Next plan reconciles. Never promise rollback.

🚀 SENIOR DEPTH

Failed apply guarantees nothing about the world: inspect state AND reality, then let the next plan heal. Hand-typed -auto-approve in prod is how the interruption finds you.

🚩 RED FLAG

"All-or-nothing rollback" or "retry resumes where it stopped" — both fictions; re-entry is a fresh plan.

Q5? fmt vs validate in CI — which gate does what?

Inputs, Precedence & Types

Q1 🧯 ? Defaults, TF_VAR, terraform.tfvars, -var all set — who wins?

✓ SAFE (JUNIOR → MID)

-var wins. Ladder top-down: CLI flags → files → env vars → defaults. Value surprise? Strip top-down; the stray flag is the prime suspect.

🚀 SENIOR DEPTH

Env sits BELOW files, not above — the classic inversion. Per-team sizes come from variables with per-caller values, never from copied repos or console edits.

🚩 RED FLAG

"Env beats files" or "defaults override" — reciting the ladder backwards fails the easiest senior probe there is.

Q2? terraform.tfvars vs *.auto.tfvars — the difference?

✓ SAFE (JUNIOR → MID)

terraform.tfvars is the single auto-loaded values file; *.auto.tfvars are layered overlays loaded lexically in name order.

🚀 SENIOR DEPTH

Explicit -var-file vs automatic layering: overlays stack (base.auto.tfvars → prod.auto.tfvars), so later names win. One file can't layer; overlays can't be explicit.

🚩 RED FLAG

Secrets in either file committed to git — layering doesn't launder credentials.

Q3? What gets committed: real tfvars or the example?

✓ SAFE (JUNIOR → MID)

Template shared, secrets local: commit *.tfvars.example, ignore real *.tfvars.

🚀 SENIOR DEPTH

Full gitignore canon: ignore .terraform/, tfstate (+backup), real tfvars. Commit lock file + versions.tf — reproducibility is the half interviewers grade.

🚩 RED FLAG

"Commit tfvars so the team stays in sync" — sync via the example, never via values.

Q4? TF_VAR_tags for a map variable — exact syntax?

✓ SAFE (JUNIOR → MID)

Single-quoted HCL: TF_VAR_tags='{Env="prod"}' — shell-safe quoting so braces survive.

🚀 SENIOR DEPTH

Same rule for objects. Bare braces split on spaces, double quotes interpolate — both corrupt before Terraform ever sees the value.

🚩 RED FLAG

Unquoted complex TF VAR and "Terraform parsed it wrong" — the shell not Terraform.

Q5? TF_VAR_count="three" on a number var — result? And for each over a bare list?

Count, for_each & Providers

THE ONE LESSON

Under **count**, the index is the identity — remove the middle and everything after shifts. Under **for_each**, the key is the identity — edits touch one instance only. Say that sentence and half this zone answers itself.

Q1 🔥 count vs for_each — when do you use which?

✓ SAFE (JUNIOR → MID)

count clones N identical copies addressed **web[0]**, **web[1]**; for_each stamps one instance per key, addressed **web["blue"]**. Copies → count; named things → for_each.

🚀 SENIOR DEPTH

Index identity vs key identity: deleting **web[1]** under count renames **web[2] → web[1]** and rebirths the tail; under for_each only that key is destroyed. Only keys survive reordering — that is why sets/maps, never lists, feed for_each.

🚩 RED FLAG

"They're interchangeable" — signals you've never removed an item from the middle of a counted fleet and watched the cascade in the plan.

Q2 🔥 for_each over a bare list fails. Fix?

✓ SAFE (JUNIOR → MID)

for_each needs a **set or map**. Wrap it — **for_each = toset(var.list)** — or build a key map. Lists re-index, so they can't supply stable identity.

🚀 SENIOR DEPTH

toset() casts list → stable set so each *value* becomes its own key; for richer objects, a map gives you **each.key / each.value** with per-field types. Duplicates collide at plan time — that error is the identity check working.

🚩 RED FLAG

"Just use count instead, always" — dodges the identity problem instead of solving it; the next edit rebirths the fleet.

Q3 count = 3 → count = 2. What gets destroyed?

✓ SAFE (JUNIOR → MID)

web[2] — the tail. Count removes from the end; web[0] and web[1] are untouched.

🚀 SENIOR DEPTH

The corollary is the real interview answer: removing from the *middle* (targeted destroy of web[1]) shifts every later index and rebirths the tail. for_each keys don't shift — the exam's core repetition lesson, and the reason fleets use keys.

🚩 RED FLAG

"web[0]" or "all three" — guessing reveals you've never read a count-shrink plan line by line.

Q4 How do you make a resource conditional, and read it safely?

✓ SAFE (JUNIOR → MID)

count = var.on ? 1 : 0 toggles one instance; read it with **one(x[*].attr)** — null when off. Never index **x[0]** directly: it explodes when the resource is absent.

🚀 SENIOR DEPTH

Consume, Author, Version

THE CONTRACT

Every module call is three parts: **source** (where the code lives), **inputs** (what you feed it), **outputs** (what it hands back). A fourth invented part — credentials inside, state inside — always marks the distractor.

Q1 🔥 How does a stack consume module outputs?

✓ SAFE (JUNIOR→MID)

As a normal reference — `module.vpc.private_subnets[0]` is a value like any other. Root calls children; children expose outputs the root consumes. Outputs flow upward only.

🚀 SENIOR DEPTH

Outputs are the module's public API: they turn internal resource attributes into typed, documented values downstream code, other modules, and CI can rely on. "Display-only" or "console-only" answers confuse **output** blocks with **terraform output** reads.

🚩 RED FLAG

"Only via remote state" — remote state wires *stacks*, not modules. Reaching across stacks for what an output already hands you is architecture by accident.

Q2 🔥 Pin a registry module to 5.x. Exact block — and the risk if version is missing?

✓ SAFE (JUNIOR→MID)

source + version = "5.1.0" (or `~> 5.0`). Source says WHERE, version says WHICH. Missing version floats — a future major can break runs that changed nothing.

🚀 SENIOR DEPTH

Pinning without a source is a wish; sourcing without a pin is a gamble. Local paths take *no* version key (no registry to resolve); git URLs pin commits, not registry versions. For a bug-fix bump, pin the patched minor explicitly — review majors by hand.

🚩 RED FLAG

"The registry pins automatically" — it doesn't. That sentence has caused more surprise majors than any other in this ecosystem.

Q3 Which module sources are valid?

✓ SAFE (JUNIOR→MID)

Local paths, public/private registry, git URLs, HTTP/S3 archives — not just the public registry, not Docker Hub. Installed copies land in `.terraform/modules` after init.

🚀 SENIOR DEPTH

Init collision errors come from `.terraform/modules`, never the registry — two copies of one source collide locally; the registry is remote and read-only. New module + weird behavior → "did you init?" — apply only runs installed code.

🚩 RED FLAG

"Only the public registry" — rules out every enterprise pattern (private registry, mono-repo local paths) in one breath.

Q4 Should child modules declare provider blocks?

✓ SAFE (JUNIOR→MID)

Avoid it. **inherit from the root**. Provider config flows DOWN, never as a module input; required providers declare the

Gates, Locks & Safe Surgery

THE FIVE GATES

fmt -check → **init** → **validate** → **apply** → **plan (idempotent, no drift)**. Style, wiring, syntax, execution, proof. Name all five and the interviewer knows you've shipped, not just studied.

Q1 🔥 CI must fail on unformatted HCL without rewriting. Command — and validate's job?

✓ SAFE (JUNIOR → MID)

terraform fmt -check — exit 1 on any unformatted file, zero writes. **validate** checks syntax + consistency with zero cloud contact: no credentials, no network.

🚀 SENIOR DEPTH

Gate = -check (rewrites belong on laptops); format-on-save + CI -check covers prevention *and* enforcement. validate parses, resolves references, type-checks — anything needing AWS (refresh, diffs, costs) is outside its universe. **-recursive** covers subfolders; **-diff** previews.

🚩 RED FLAG

"Bare fmt in CI" — gates must not mutate. A gate that rewrites is a race with the checkout it runs on.

Q2 Second apply starts while the first holds the lock. What happens?

✓ SAFE (JUNIOR → MID)

It **fails with a lock error naming the holder and lock ID** — no parallel writes, no merges. Locks are protective, not punitive.

🚀 SENIOR DEPTH

One lock table per env buys parallel applies across envs at the cost of more tables to guard; delete the table and applies run *unlocked* — last writer wins, silently. Lock fights mean a second writer: coordinate, don't force.

🚩 RED FLAG

"Re-run with -auto-approve until it passes" — retries collide again without coordination, and -auto-approve near a lock fight is how two writers become zero states.

Q3 🔥 Lock held by a dead CI runner. Exact recovery?

✓ SAFE (JUNIOR → MID)

Confirm death, then force-unlock by ID, then prove health. Verify no live process holds it first — stale crash, then release.

🚀 SENIOR DEPTH

Force under a *live* holder corrupts state for both runners — the cardinal sin. Deleting the DynamoDB row destroys locking for everyone; restoring backups fixes state, and the lock isn't state. Sequence it: prove dead → unlock by ID → plan clean before any apply.

🚩 RED FLAG

"Delete the lock row by hand" or "destroy and recreate the bucket" — surgery for a splinter, with blast radius across every team sharing the backend.

Q4 Why plan -out=tfplan + apply tfplan instead of bare apply?

✓ SAFE (JUNIOR → MID)

apply tfplan executes **EXACTLY** the saved plan — reviewed == executed, no drift between review and run. Bare apply re

Workspaces, TF_VAR & the ALB Stack

THE MENTAL MODEL

Workspaces isolate **state, not code** — one configuration, N state files under `terraform.tfstate.d/<name>/`. Per-env differences ride variables and `terraform.workspace` lookups, never copied folders.

Q1 🔥 workspace show vs list — and which workspace can't be deleted?

✓ SAFE (JUNIOR → MID)

show prints only the active name ("where am I"); **list** prints all names. **default** is implicit and permanent — it survives every delete attempt.

🚀 SENIOR DEPTH

Deleting default would strand unscoped state, so migrate away from it instead. A brand-new workspace starts with *empty* state — first plan wants to create everything, which is exactly the surprise that bites during demos.

🚩 RED FLAG

"Workspaces isolate code / providers / regions" — they isolate state. Regions live in provider blocks, code is shared; that's the entire point.

Q2 What is terraform.workspace in code, and what's the idiom?

✓ SAFE (JUNIOR → MID)

A read-only string of the active name — the key into per-env maps: `locals { env = terraform.workspace }`, then lookup sizes, CIDRs, tags. Derive, don't repeat.

🚀 SENIOR DEPTH

Single source per env: one map keyed by workspace beats conditionals scattered across resources. It never lists, versions, or addresses anything — backends come from blocks, resources from state, versions from the lock file.

🚩 RED FLAG

"Use it for backend config" — backend blocks can't use `var.*` or workspace expressions; backend args come from `-backend-config` files.

Q3 🔥 Defaults, TF_VAR, terraform.tfvars, -var all set region. Winner — and the TF_VAR map gotcha?

✓ SAFE (JUNIOR → MID)

-var wins. Ladder top-down: CLI flags → files (incl. `*.auto.tfvars` overlays) → `TF_VAR_*` env → defaults. Value surprise? Strip layers top-down; the stray flag is the prime suspect.

🚀 SENIOR DEPTH

`TF_VAR` maps need **single-quoted HCL** (shell-safe object syntax — bare braces split, double quotes interpolate). A variable with no default and nothing provided *prompts* — which hangs CI; always feed it via `-var/env/file`.

🚩 RED FLAG

"Env vars beat files" — inverted ladder. Misordering the ladder means debugging values in exactly the wrong direction.

Q4 tfvars discipline: what gets committed, what never does?

✓ SAFE (JUNIOR → MID)

Commit `*.tfvars.example`, ignore real `*.tfvars`, `terraform/`, local state + `TOC` `*.auto.tfvars` loads automatically vs explicit `-var-file` — know which magic is active.

TFC, Sentinel & Secrets

THE GOVERNANCE TRIANGLE

Blessed code (private registry) + locked versions (pins) + enforced rules (Sentinel) + secret-free repos (env/Vault). Miss one side and the other three can't save you.

Q1 TFC vs TFE — and what does a TFC workspace bind?

✓ SAFE (JUNIOR → MID)

TFC = SaaS hosted by HashiCorp; TFE = self-hosted in your boundary. A TFC workspace binds **config + variables + state + run history** — run history lives there, not on laptops.

🚀 SENIOR DEPTH

Heavy teams outgrow the free tier into paid runners/agents: remote operations run on your infra via agents, so plans execute identically for every engineer. "Just the CLI binary" or "display only" answers miss that the workspace *is* the collaboration unit.

🚩 RED FLAG

"TFE is older Terraform" — confuses a deployment model with a version. Says you've never had to place a compliance boundary.

Q2 🔥 PR opened → plan posted back → merge → apply. Name it and defend it.

✓ SAFE (JUNIOR → MID)

VCS-driven runs. Remote runs fire on VCS events; the speculative plan lands as a PR comment with zero state risk; merge gates the apply.

🚀 SENIOR DEPTH

Speculative plans analyze the plan remotely — same engine feeds cost estimates on the PR. Golden rule: infra change without a PR is an incident report waiting for a timestamp. Console runs, local-exec, and state mv are unrelated machinery.

🚩 RED FLAG

"We apply from laptops, it's faster" — speed that skips review, audit trail, and identical execution is the thing governance exists to kill.

Q3 🔥 Sentinel: mandatory vs soft-mandatory vs advisory?

✓ SAFE (JUNIOR → MID)

Missing cost tags, no override → **mandatory**: never passes. **Soft-mandatory**: passes only with an override by someone holding the right; **advisory**: warns, never blocks.

🚀 SENIOR DEPTH

The axis is *override possibility* — permitted vs never — not speed or language. Design rules so the common path flows (advisory for new standards, soft-mandatory during rollout, mandatory once the org can comply), or engineers route around the policy system entirely.

🚩 RED FLAG

"Everything mandatory" — a policy set nobody can satisfy is a policy set everybody bypasses. Governance that blocks work gets uninstalled.

Q4 Where do secrets actually live — state, tokens, CI creds?

✓ SAFE (JUNIOR → MID)

2 AM Triage Scenarios

🔧 HOW TO USE THIS PAGE

Read the symptom, say your first 3 commands *before* reading the triage. Interviewers score the order: observe → isolate → fix. Jumping to the fix first is the junior tell.

W1 init → AccessDenied / hangs. 2 AM, pipeline red.

✅ TRIAGE

floci status → check **provider.tf** endpoints → confirm dummy creds exported. Cause: emulator not running or endpoint wrong.

🚀 FIX + PREVENT

floci start, verify endpoint lines, re-init. Prevent: health-check the emulator *before* init in the wrapper script so CI fails fast with a useful message.

W2 plan hangs forever — no output, no error.

✅ TRIAGE

curl the health endpoint → **floci status** → diff **provider.tf** endpoint lines vs the live port. Cause: endpoint points at a dead port / missing line.

🚀 FIX + PREVENT

Fix the endpoint, re-plan. Prevent: fail-fast connectivity probe at the top of run scripts; never let plan be the first thing that touches the network.

W3 🔥 apply is creating REAL AWS resources — real bill accruing.

✅ TRIAGE

Ctrl-C / cancel the run → **env | grep -i aws** (which profile/creds are live?) → inspect endpoint lines in **provider.tf**. Cause: endpoint missing + real profile active.

🚀 FIX + PREVENT

Destroy what escaped, confirm endpoints + dummy creds, never mix a real profile into emulator runs. Prevent: explicit endpoint assertion + deny-real-account guard before every non-prod apply.

W4 apply ok, but aws s3 ls shows nothing.

✅ TRIAGE

Re-run ls **with the same --endpoint-url + region + test creds** → compare URL against **provider.tf** → **terraform show** to confirm state recorded it.

🚀 FIX + PREVENT

Align the CLI invocation with the provider config — the resource is real, you're looking through the wrong window. Prevent: one env file sourced by both Terraform and CLI wrappers.

W5 🔥 State lost: destroy says "No changes" but the bucket exists. Then: state file deleted → apply?

✅ TRIAGE

terraform state list (empty?) → check backend config / state path → **state pull** for any surviving snapshot. Cause: two backend configs, or the ledger is gone.

Can You Ship Safely?

RULES

Answer aloud, 2 min each. Interviewer: award **0 / 1 / 2** per question (miss / safe / safe+depth). No peeking at rungs until the timer ends.

M1 CI must fail on unformatted HCL without rewriting. Command?

✓ MODEL (2 PTS)

fmt -check — exit 1, zero writes. Bare fmt rewrites (laptops), validate checks syntax (different job), plan previews infra.

M2 What does terraform.tfstate primarily let Terraform do?

✓ MODEL (2 PTS)

Map configuration to real-world objects — track IDs/metadata, compare desired vs actual, cache for fast plans. Delete it → duplicates.

M3 validate vs plan — what does each need and prove?

✓ MODEL (2 PTS)

validate: syntax + consistency, **no cloud, no credentials**. plan: needs init/providers/refresh — previews the diff, changes nothing.

M4 workspace show prints...? And which workspace is immortal?

✓ MODEL (2 PTS)

Only the **active name** (list shows all). **default** can't be deleted — migrate away, never remove.

M5 TF_VAR, tfvars file, -var all set. Who wins?

✓ MODEL (2 PTS)

-var > files > TF_VAR > defaults. Strip top-down; the stray CLI flag is the prime suspect in value surprises.

M6 count 3→2 destroys...? And the middle-removal corollary?

✓ MODEL (2 PTS)

web[2], the tail. Middle removal shifts later indexes → tail rebirths. for_each keys don't shift.

M7 Rename web→app, zero rebuild. Retire legacy without deleting it.

✓ MODEL (2 PTS)

state mv renames the binding (empty plan). **state rm** forgets (object lives); **destroy** kills it. Taint is destruction — opposite of rename.

M8 Sensitive values in state are...? Where does login store its token?

✓ MODEL (2 PTS)

Plaintext — bucket + IAM lockdown, Vault/SSM. Token → **credentials.tfrc.json** in home, never the repo.

Can You Design & Survive?

► SENIOR BAR

Safe answers earn nothing here — every question is scored on **mechanism + WHY + blast radius**. "It works" is 0 points; "it works because, and when it breaks we..." is full marks.

S1 Design fleet addressing for 50 subnets that churn monthly. count or for_each — defend with failure mode.

🚀 MODEL (2 PTS)

for_each over a keyed map (AZ/name keys). Count churn rebirths the tail monthly; keys isolate each change. Duplicates must collide loudly at plan.

S2 Org consumes 40 registry modules. Versioning + registry strategy?

🚀 MODEL (2 PTS)

source + version pins everywhere (review majors by hand); blessed copies in the **private registry**; missing pins float into surprise majors. Lock file committed.

S3 Dead CI runner holds the lock; second team is blocked. Walk the recovery.

🚀 MODEL (2 PTS)

Prove death → **force-unlock by ID** → plan clean before apply. Live-holder force = corruption for both. Long-term: per-env tables + CI run TTLs.

S4 Design the PR-to-prod pipeline: speculative plans, policy, cost.

🚀 MODEL (2 PTS)

VCS-driven runs (speculative plan on PR, zero state risk) → **Sentinel** (advisory→soft-mandatory→mandatory rollout) → **remote-run cost estimate** → merge-gated apply tfplan.

S5 Secrets end to end: state, outputs, CI logs. Where does each leak, and the fix?

🚀 MODEL (2 PTS)

State = **plaintext** (bucket+IAM, Vault/SSM); **sensitive** masks display only (-raw to logs still leaks); **ephemeral** for same-apply-only reads; CI creds via env/OIDC, never the repo.

S6 Emulator apply panics on EC2 stubs (code 48). Triage live, then defend keeping the blocks.

🚀 MODEL (2 PTS)

Confirm ALB tiers landed, identify the stub limit, **exclude EC2 targets from emulator applies** — blocks stay for production. Fidelity table so nobody "fixes" correct IaC.

S7 Console drift on a security group. Detect → decide → reconcile, modern commands.

🚀 MODEL (2 PTS)

Refresh surfaces it as ~; **plan -refresh-only** previews (bare refresh is legacy); apply re-asserts code — or **apply -refresh-only** if the console was right.

S8 Adopt a hand-built VPC vpc-0ab1 with zero downtime. Exact sequence.

🚀 MODEL (2 PTS)

Write the matching block → **import** → plan until empty. Import binds identity only, never generates config; **state pull**

24 One-Line Q → A

⚡ DRILL

Cover the right column. One breath per answer. Any hesitation → back to that zone tonight.

Question	Answer
Second apply, no edits — plan says?	0/0/0, "No changes" — idempotence, the healthy steady state.
First command in a fresh clone?	init — providers + modules + backend + lock file.
File that guarantees identical providers?	.terraform.lock.hcl — commit it; ignore .terraform/.
File that must NEVER be committed?	terraform.tfstate — plaintext secrets + keys to everything.
validate proves what, touching what?	Syntax + consistency, zero cloud contact .
CI style gate without rewriting?	fmt -check — exit 1, no writes.
Write → ? → ? → ?	write → init → plan → apply .
Reviewed == executed — how?	plan -out=tfplan → apply tfplan .
for_each accepts...?	Set or map — bare lists wrap with toset().
web["blue"] implies...?	for_each keys — string-key identity.
count 3→2 destroys...?	web[2] — the tail; middle removal cascades.
Safe read of a 0/1 counted resource?	one(x[*].attr) — null when off.
Nested-block repetition lives where?	dynamic blocks — data in, stanzas out.
Cross-region block selects how?	provider = aws.eu — aliased instance.
Registry module pin = ?	source + version — one without the other is a gamble.
Installed modules land where?	.terraform/modules — collisions happen there.
show vs list (workspaces)?	show = active name; list = all names.
Workspace you can't delete?	default — implicit and permanent.
-var vs TF_VAR vs files?	-var > files > TF_VAR > defaults .
Rename with zero rebuild?	state mv — new address, same object.
rm vs destroy?	rm forgets (lives); destroy deletes (dies).
Dead-holder lock recovery?	Confirm death → force-unlock by ID → plan clean.
PR → posted plan → merge → apply?	VCS-driven runs — speculative, zero state risk.
Mandatory vs soft-mandatory?	Override: never vs permitted — advisory only warns.

Checklist & Trap Radar

✓ NIGHT-BEFORE CHECKLIST

Run it in order. Sleep beats cramming after box 6.

Pre-flight (do tonight)

- ☐ Five-gate pipeline cold: **fmt -check** → **init** → **validate** → **apply** → **plan (no drift)**.
- ☐ One-liners ready: 0/0/0, lock.hcl, toset(), state mv, -var ladder, VCS-runs.
- ☐ Numbers memorized: dev **vpc-5e7eebfd** 6-added · staging/prod 22-added · EC2 stub code 48.
- ☐ Two stories rehearsed aloud: one triage (W1–W8), one design (fleet or pipeline).
- ☐ Laptop rules: no -auto-approve by hand, no real profile near emulator dirs, creds via env.

Morning-of (before you leave)

- ☐ Say the identity sentence once: *"Index is identity under count; key is identity under for_each."*
- ☐ Say the governance triangle once: *blessed code + locked versions + enforced policy + secret-free repos.*
- ☐ Pick your closer: one incident you survived, 30 seconds, with the prevention you added.

🔗 TRAP RADAR — 6 RULES

Trap	Rule
1 · EXCEPT / NOT stems	Invert first: cover the answers, ask "which one <i>doesn't</i> belong" — e.g. state purposes EXCEPT compiling HCL . Most misses are right answers to the wrong question.
2 · Workspace homonyms	Three different things share the name: CLI workspaces (state isolation), terraform.workspace (a string key), TFC workspaces (config+vars+state+runs). Ask which one before answering.
3 · -var precedence	Ladder: -var > files > TF_VAR > defaults . Value surprise? Strip top-down — the stray flag is always the prime suspect, env never beats files.
4 · Sensitive-display myth	sensitive masks, never encrypts — plaintext stays in state; -raw to logs still leaks. Real protection is bucket+IAM, Vault/SSM, and ephemeral for same-apply reads.
5 · Count-vs-key identity	[0] vs ["az"] : only keys survive reordering. Any stem with removal/reorder/shrink → answer in identity terms, then name the cascade (count) or the isolation (for_each).
6 · Skipped-plan stems	"Straight to apply", "no review", "laptop -auto-approve" → the answer is always the saved plan + gate : plan -out, review, apply tfplan, VCS-driven runs in prod. Speed is never the control.

💡 THE CLOSER

"Safe gets me to the next question; senior ends the topic." Climb every question until the interviewer nods and moves on — the nod is the deploy confirmation.